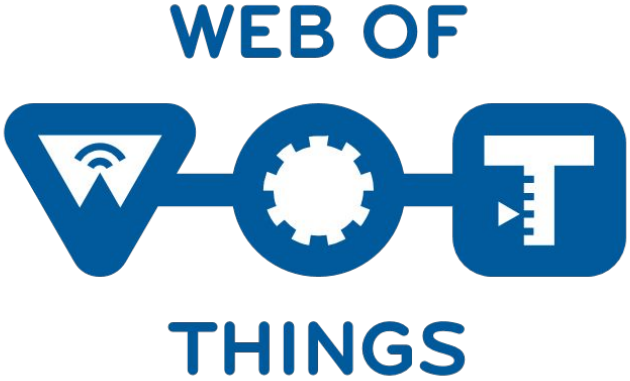


WoT Scripting API

W3C WoT TPAC, October 2026

Table of Contents

- Publication Plans
- Next Charter Topics
- Implementations

Publication

Last publication: https://www.w3.org/TR/wot-scripting-api/



Draft at https://w3c.github.io/wot-scripting-api/ with the following changes:

- Fetch TD vs. Request a TD (Clarifications)
- ActionInteractionOutput interface
- Error handling and additional responses support
- Algorithm clarifications/improvements

ActionInteractionOutput

This interface exposes functions which will allow cancelling asynchronous actions and query the status of a long running action.

WebIDL

```
/**
 * Note: retrieving the result of an action via the implicit InteractionOutput interface
 * will only work after the action has been completed
 */
[SecureContext, Exposed=(Window,Worker)]
interface ActionInteractionOutput : InteractionOutput {
  Promise<InteractionOutput> query(
    optional InteractionInput params = {},
    optional InteractionOptions options = {});
  Promise<undefined> cancel(
    optional InteractionInput params = {},
    optional InteractionOptions options = {});
};
```

Error handling

Represents an error that occurred during a WoT Interaction with additional diagnostic information from the Thing in the form of response data associated with an AdditionalExpectedResponse defined in the Thing Description.

`InteractionError` is a DOMException derived interface. As recommended for such interfaces, it is a serializable object whose serialization and deserialization steps preserve the additional information it carries.

WebIDL

```
[SecureContext, Exposed=(Window,Worker), Serializable]
interface InteractionError : DOMException {
  constructor(optional DOMString message = "", optional InteractionErrorOptions options = {});
  readonly attribute AdditionalExpectedResponse? response;
  readonly attribute DataSchema? schema;
  Promise<ArrayBuffer> arrayBuffer();
  Promise<DataSchemaValue> value();
};

dictionary InteractionErrorOptions {
  ArrayBuffer data;
  AdditionalExpectedResponse response;
  DataSchema schema;
};
```

Error & Response Handling Flow

1. Got Error payload?

Check Response:

Is the response null or undefined?

If True:

Map to suitable DOMException or OperationError.

➔ **Return Error & Stop**

2. Match Additional

Iterate Entries:

For each responseEntry in additionalResponses:

If Match & Error:

Matches binding & success is false/undefined.

Extract payload data & schema.

➔ **Return InteractionError**

3. Binding Check

No Match Found:

No matching entry in additional responses.

If Binding Failed:

Protocol binding indicates a failure.

➔ **Map & Return DOMException**

4. Default Fallback

Otherwise:

No specific error mapped by previous steps.

Fallback Action:

Default to generic operation failure error.

➔ **Return OperationError**

https://w3c.github.io/wot-scripting-api/#the-handle-interaction-error-response-algorithm

Thing Description 1.1 parity

Before publication, we plan to solve the remaining issue:

- Cover AdditionalExpectedResponses as success responses

Once done, we are "almost" feature complete in comparison with Thing Description 1.1 :

- The major missing part is still how to deal with *All* or *Multiple* operations, which lack information about input/output schemas.
  A possible solution is similar to what we resolved for Async actions -> do not assume any payload type and let the application use a particular format.

Publication Timeline

- Final document to be reviewed by the end of November
- 2-week review period
- Publication within 2026

Next Charter Topics

Issues with

- “for next iteration”
- “wait -for-td”
- “Spec-improvement”
- “enhancement”
- “API-improvement”
- “Use case”
- Fleet management/API
- …

Goals: Keep the APIs aligned with future Thing Description revisions and keep the same abstraction level of the Thing Description interaction model (e.g., Common Definitions).

Bi-weekly calls are enough and allow us to keep focus on TD topics

New TD Toolchain and Its Impact on Us

- The Scripting API relies on the JSON schema defined by the Thing Description (TD)
- The TypeScript definitions are automatically generated from this JSON schema and published on NPM for use by implementations such as Eclipse Thingweb
- The new TD toolchain models WoT resources in LinkML and generates the related artifacts from the same schema modules
- One of these generated artifacts is the JSON Schema. In theory, the toolchain could also generate TypeScript definitions, but the current results are not yet satisfactory
- In PR#609, we examined how the slightly revised JSON Schema affects the types used in our implementation.
  - Since the new toolchain is being developed for TD 2.0, incompatible changes may occur and could affect our existing implementation

API implementations

We are collecting more implementations of Scripting API under the “thingweb” umbrella

- node-wot (TypeScript), with some experiments like
  - CSV Datamapping
  - Data Mapping (extracts specific values from a Thing's response)
  - Connection handling
- dart_wot (Dart)
- kotlin-wot (Kotlin)
- wotpy (python) → published recently

- Many tools around W3C WoT specifications
  - playground
  - td-tools
  - node-red
  - ui-wot
  - Domus-tdd-api

- Others
  - .NET developed by TU Munich: https://github.com/tum-esi/WoT.Net
  - …

Call for help

We need more

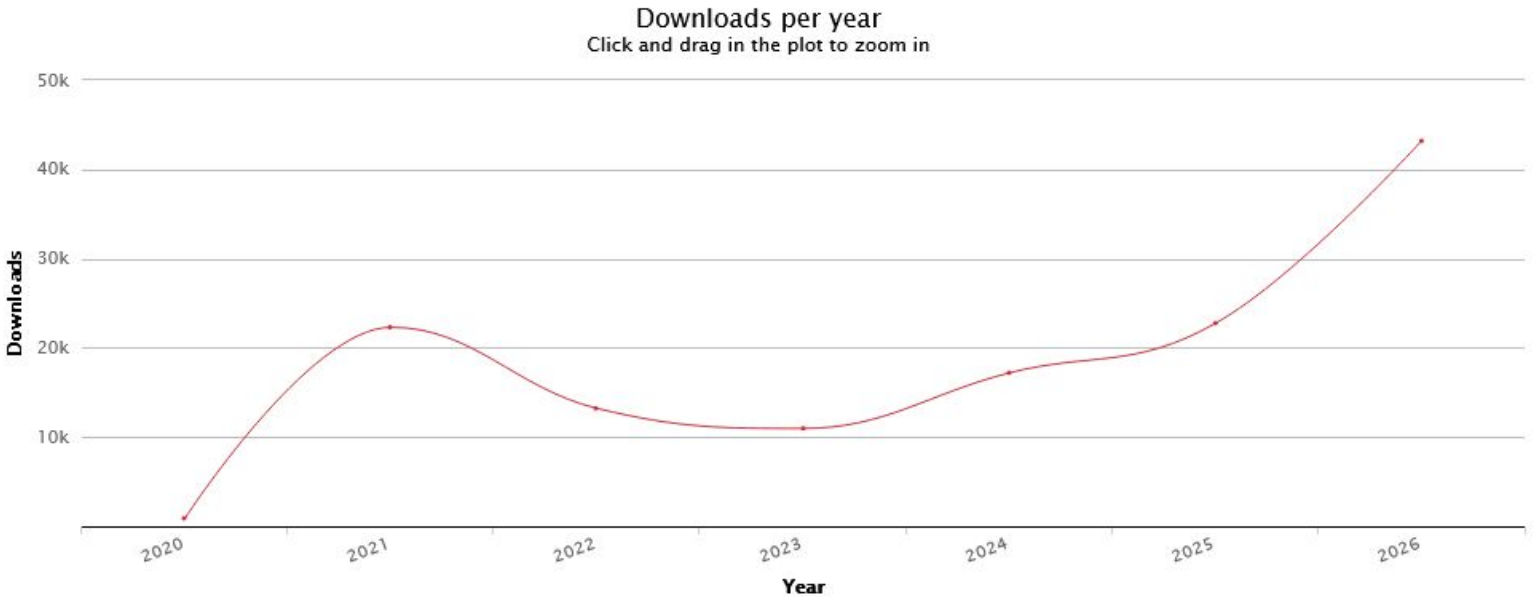
- participation in our calls
- contributions and reviews on our pull requests (PRs)

Interested in shaping the future of the WoT Scripting API?

Join us and help develop the next generation of the WoT Scripting API!

node-wot statistics

https://npm-stat.com/charts.html?package=@node-wot/core



Steady increase since 2023.